

Digital Architects of Africa: The PaveWay Mentorship Program

"Building the Operating System for the Continent"

- **Mentor:** Mbiarrambang Alain, Solutions Architect @ PaveWay
- **Schedule:** Mondays & Wednesdays (2 Hours/Session)
- **Duration:** 16 Weeks (32 Sessions) + 4 Week Capstone
- **Focus:** Full Stack (Next.js/React) | Python for Data | GCP DevOps
- **Goal:** To transition students from "Coders" to "Solutions Architects" capable of building the next Stripe, DoorDash, or Google for Africa.

1. The Philosophy: "Code is a Tool, Solution is the Goal"

Most bootcamps teach syntax. This mentorship teaches **Architecture**.

- **The Steve Jobs Standard:** It's not done until the user experience is magical.
- **The Elon Musk Standard:** First Principles Thinking. Don't copy; deconstruct the problem to its core truth.
- **The PaveWay Standard:** Context matters. A solution in San Francisco might fail in Douala if it doesn't account for trust, bandwidth, or culture.

2. Program Roadmap (High Level)

Phase	Focus	Tech Stack	Goal
Phase 1: The Architect's Mindset	Weeks 1-2	Whiteboards, Figma, Logic	Learn to see problems & design systems before writing code.
Phase 2: The	Weeks 3-6	Node.js, Express,	Building the API &

Engine (Backend)		MongoDB	Logic Layer. Secure, fast, scalable.
Phase 3: The Interface (Frontend)	Weeks 7-10	Next.js (React), Tailwind	Building "Trust" through Performance. SSR, SEO & Server Actions.
Phase 4: The Brain (Data)	Weeks 11-12	Python, Pandas, Jupyter	Integrating Data Science. Making the app "smart."
Phase 5: The Infrastructure (DevOps)	Weeks 13-14	Google Cloud, Docker, CI/CD	Deploying to the world. Monitoring & scaling.
Phase 6: The "Unicorn" Capstone	Weeks 15-16	Full Stack Integration	Building a simplified PaveWay ecosystem product.

3. Detailed Session Guide

Phase 1: The Architect's Mindset (Weeks 1-2)

Focus: Creative Thinking, Problem Solving, System Design.

Week 1: Deconstruction & The "Why"

Session 1 (Monday): The African Problem Space

- **Theme:** "Why are we here?" & First Principles Thinking.
- **Instructor Lead (45m):**

- Intro to PaveWay's Mission: Solving the "Trust Gap."
- **Case Study:** Why does "Pay on Delivery" exist? (Answer: Lack of trust). How do we architect trust?
- *Exercise:* "The 5 Whys." Pick a local problem (e.g., "Traffic in Douala"). Ask "Why" 5 times until you hit the root cause.
- **Student Activity (1h):**
 - Break into pairs. Identify one "Broken" system in their community.
 - Draft a "Problem Statement" (not a solution).
- **Homework:** Read PaveWay's "North Star" Manifesto (Exec Summary). Identify 3 tech architectural decisions PaveWay made (e.g., using US Entity for USD, using local logistics for escrow).

Session 2 (Wednesday): System Design & "Napkin Architecture"

- **Theme:** Drawing the solution.
- **Instructor Lead (45m):**
 - Intro to System Diagrams: Client -> API -> Database.
 - The concept of "State."
 - **Live Demo:** Draw the architecture of a "Momo Kiosk" app on a whiteboard.
- **Student Activity (1h):**
 - Take the "Problem" from Monday.
 - Draw the High-Level Architecture (User -> Mobile App -> Cloud -> Data).
 - **Constraint Challenge:** "What if the internet cuts off?" (Introduce Offline-First Architecture).

Week 2: The Architect's Tools

Session 3 (Monday): Algorithms & Logic (Without Code)

- **Theme:** Thinking in flowcharts.
- **Instructor Lead:** Pseudo-code. How to write logic before syntax.
- **Activity:** Write the pseudo-code for a ride-sharing dispatch algorithm.

Session 4 (Wednesday): Environment Setup & The "Hello World" of Architecture

- **Theme:** Professional Setup.
- **Tech:** VS Code, Git/GitHub, Node.js installed.

- **Activity:**
 - Setup the "Monorepo" structure.
 - Create a simple README.md describing their Capstone idea.
 - *Git Flow*: meaningful commit messages.

Phase 2: The Engine - Backend Engineering (Weeks 3-6)

Focus: Node.js, Express, MongoDB (The Core Backend).

Week 3: HTTP & APIs

Session 5 (Monday): The Language of the Web (HTTP/JSON)

- **Concept:** Request/Response cycle. REST principles.
- **Code:** Spin up a raw Node.js server.
- **PaveWay Context:** How Pursar talks to the bank.

Session 6 (Wednesday): Express.js Framework

- **Code:** Refactor Node server to Express.
- **Task:** Build 3 endpoints: GET /users, POST /login, GET /balance.

Week 4: Data Persistence (MongoDB)

Session 7 (Monday): NoSQL vs SQL

- **Concept:** Why Mongo? (Flexibility for startups). Designing Schemas.
- **Activity:** Design the Schema for a "PaveWay Merchant" (Name, Location, Inventory).

Session 8 (Wednesday): Mongoose & CRUD

- **Code:** Connect Express to MongoDB Atlas. Implement Create, Read, Update, Delete.

Week 5: Authentication & Security

Session 9 (Monday): JWT & Security

- **Concept:** Stateless authentication. "Never trust the client."
- **Code:** Implement Login/Register with bcrypt (hashing passwords) and JWT.

Session 10 (Wednesday): Middleware & Protection

- **Concept:** Protecting routes. Rate limiting (DDOS protection).

- **Code:** Create an authMiddleware function.

Week 6: Advanced Backend Logic

Session 11 (Monday): Business Logic Separation

- **Concept:** Controllers vs. Services. Keep routes clean.
- **Code:** Refactor code into Service layers.

Session 12 (Wednesday): Testing APIs

- **Concept:** Postman/Insomnia usage.
- **Challenge:** "Break the API." Students try to hack each other's APIs.

Phase 3: The Interface - Next.js & React (Weeks 7-10)

Focus: Modern Frontend Architecture, SSR, and Performance.

Week 7: The React Mental Model & Next.js

Session 13 (Monday): Why Next.js? (The Full Stack Framework)

- **Concept:** React (Library) vs Next.js (Framework). Client vs Server Components.
- **Code:** `npx create-next-app`. Understanding the App Router (/app directory).
- **PaveWay Context:** Why SSR (Server Side Rendering) matters for low-end devices in Africa.

Session 14 (Wednesday): Components & Props

- **Concept:** Reusability. JSX. Passing data.
- **Code:** Build a reusable Button and TransactionCard component.

Week 8: Data Fetching & Rendering Strategies

Session 15 (Monday): Server Side Rendering (SSR) vs Static (SSG)

- **Concept:** When to render? (Build time vs Request time).
- **Code:** Fetching data inside a Server Component (async components).

Session 16 (Wednesday): Client Interactivity

- **Concept:** Using useClient for interactivity (clicks, state).
- **Code:** Building a dynamic "Currency Converter" that runs in the browser.

Week 9: Forms & Mutations (Server Actions)

Session 17 (Monday): Server Actions

- **Concept:** No more manual API calls? Mutating data directly from the frontend code.
- **Code:** Build a "Send Money" form using Server Actions and formData.

Session 18 (Wednesday): Validation & Feedback

- **Concept:** Zod for validation. Handling loading states (useFormStatus).
- **Code:** Validating the money transfer inputs and showing success/error toasts.

Week 10: UX & Optimization

Session 19 (Monday): Styling with Tailwind

- **Concept:** Utility-first CSS. Mobile-first design for constrained screens.
- **Activity:** Style the PaveWay Dashboard (Dark mode support).

Session 20 (Wednesday): Optimization for Africa

- **Concept:** next/image for bandwidth saving. next/font.
- **Code:** Auditing the app with Lighthouse. Optimizing for 3G networks.

Phase 4: Intelligence - Python & Data Science (Weeks 11-12)

Focus: Adding a "Brain" to the system.

Week 11: Python for Architects

Session 21 (Monday): Python Syntax Crash Course

- **Concept:** Why Python? (Data ecosystem).
- **Code:** Python lists, dictionaries, functions. Compare with JS.

Session 22 (Wednesday): Microservices - Flask/FastAPI

- **Concept:** The "Sidecar" pattern. Node/Next handles App, Python handles Logic.
- **Code:** Create a tiny Python API that accepts data and processes it.

Week 12: Intro to Data Science

Session 23 (Monday): Pandas & Data Manipulation

- **Concept:** DataFrames. Cleaning messy data.

- **Activity:** Import a CSV of "Transaction History" and calculate total volume per region.

Session 24 (Wednesday): Visualization & Insight

- **Concept:** Matplotlib/Seaborn.
- **Activity:** Generate a chart showing user growth. Expose this chart via the Python API to the Next.js Frontend.

Phase 5: Infrastructure - DevOps on GCP (Weeks 13-14)

Focus: Deployment, Docker, Cloud.

Week 13: Containerization

Session 25 (Monday): Docker Basics

- **Concept:** "It works on my machine" -> "It works everywhere."
- **Code:** Write a Dockerfile for the Next.js App and the Python Service.

Session 26 (Wednesday): Docker Compose

- **Code:** Orchestrate the Database, Frontend, and Python service locally.

Week 14: Google Cloud Platform (GCP)

Session 27 (Monday): Intro to Cloud & Compute Engine

- **Concept:** VMs vs. Serverless.
- **Activity:** Deploy the Docker container to Google Cloud Run (Serverless).

Session 28 (Wednesday): CI/CD & Monitoring

- **Concept:** Automated deployment.
- **Code:** Set up a GitHub Action to deploy automatically on push.
- **Monitoring:** Set up Google Cloud Logging.

Phase 6: The Capstone - "The African Solution" (Weeks 15-16)

Session 29-31: Build Sprint

- **Goal:** Students build a full-stack solution.
- **Requirement:** Must solve a specific African problem (e.g., "AgriTech supply chain,"

"Informal Sector Tax Calculator," "Bus Ticketing").

- **Stack:** Next.js Frontend + Express API (or Next.js API Routes) + Python Analytics Service + MongoDB + GCP Deployment.

Session 32: The Demo Day

- **Format:** "Pitch Deck" + Live Demo.
- **Judge:** You (Alain) act as the Investor/CTO.
- **Criteria:** Code quality, Architectural soundness, Problem-Solution fit.

4. Instructor's "Cheat Sheet" for Success

The "PaveWay" Twist

Every week, dedicate 15 minutes to **"The PaveWay Reality Check"**:

1. **Week 3 (API):** "How do we handle slow internet in PaveWay's API design?" (Answer: Lightweight JSON, retries).
2. **Week 7 (Next.js):** "Why Server Side Rendering?" (Answer: Cheaper phones struggle with heavy Client Side Rendering JS bundles. We do the work on the server).
3. **Week 10 (Optimization):** "Why optimize images?" (Answer: Data costs money. Saving 1MB for a user is saving them 100 CFA).

Teaching Tools

- **Excalidraw/Whiteboard:** Use visualization constantly. Never just explain in code.
- **Code Reviews:** Every Wednesday, pick one student's code (anonymously or voluntarily) and refactor it live.
- **"Fail Fast":** Introduce bugs intentionally in your live coding (e.g., forget an await). Ask them to spot it.

Required Resources for Students

1. **Hardware:** Laptop with 8GB+ RAM.
2. **Software:** VS Code, Docker Desktop, Postman, Node.js (LTS), Python 3.9+, Git.
3. **Accounts:** GitHub, Google Cloud (Free Tier), MongoDB Atlas (Free Tier).